
MATCHMAKER

Sistema de Gerenciamento e Equilíbrio de Partidas

Autores: Henrique Bicudo / Enzo Salles / Bernardo Rodrigues

VISÃO GERAL DO PROJETO



Desenvolvimento de um sistema de matchmaking para League of Legends.



Foco em balanceamento de equipes baseado em nível de habilidade (Elo).



Implementação robusta utilizando C++ para alta performance.



Gerenciamento eficiente de filas e atribuição de funções.

LISTA E VETOR (PLAYERLIST)

- 🏗️ Uso de `std::vector<Player>` para armazenamento global e **acesso por índice** eficiente.
- ⚡ Estrutura **cache-friendly** que facilita ordenações (Quick/Merge Sort) e geração de métricas.
- 🔽 Base para iterações em relatórios, filtros de MMR e contagem de popularidade de roles.
- ↔️ Permite conversão ágil para outras views e inputs necessários ao matchmaking.

```
class PlayerList {
```

```
private:
```

```
    std::vector<Player> players;
```

```
public:
```

```
    void filterByMMR(int min);
```

```
    void generateReport();
```

```
};
```

COMPONENTES DO SISTEMA

COMPONENTE	ARQUIVO(S)	USO
Matchmaker	Matchmaker.cpp , Matchmaker.h	Criação de partidas e balanceamento.
PlayerList	PlayerList.cpp , PlayerList.h	Gerenciamento da coleção de jogadores.
Player	Player.h	Definição do modelo de dados e atributos.
Utils	Utils.cpp	Funções auxiliares para leitura e logs.

INICIALIZAÇÃO E FLUXO

- 🌱 Função `seedPlayers()` responsável pela carga inicial de dados.
- 🗄️ Geração de massa de teste para validação dos algoritmos.
- 📄 Leitura de configurações externas para parâmetros de matchmaking.
- ✅ Verificação de consistência antes do processamento das filas.



FILA E PILHA (MATCHMAKER)

Fila (MatchQueue)

FIFO — Ordem temporal de entrada.

- 🕒 Gerencia espera por partida.
- 🔄 Processamento assíncrono.
- 🛡️ Evita starvation de jogadores.

Pilha (MatchStack)

LIFO — Histórico de Partidas.

- 🕒 Guarda partidas e seus resultados.
- ↑ Trata primeiro a mais recente.
- ↶ Fluxo de resolução e desfazer.

Métodos: `push`, `pop`, `top`

ALGORITMOS DE ORDENAÇÃO

ALGORITMO	ONDE É USADO	JUSTIFICATIVA
Quick Sort	quickSortByMMR	Médio $O(N \log N)$. In-place, ideal para ordenar muitos jogadores por MMR rapidamente.
Insertion/Selection	insertionSortByRole	$O(N^2)$ geral, mas eficiente para times pequenos ($N=5$). Ordena por role.
Merge Sort	mergeSortByWinrate	$O(N \log N)$ e estável. Preserva ordem em empates, ideal para relatórios.
Bubble Sort	bubbleSortRolePopularity	$O(N^2)$. Simples e aceitável para N fixo e pequeno (ex.: 5 roles).

ATRIBUIÇÃO DE FUNÇÕES

- 🔧 Implementação do método `Matchmaker::assignRoles()` para equilíbrio.
- ⚙️ Análise de preferências primárias e secundárias dos jogadores.
- ✏️ Algoritmo de otimização para minimizar o preenchimento automático (autofill).
- ⚖️ Distribuição equitativa de funções de suporte e caçador.

TOP LANE	[ASSIGNED]
JUNGLE	[ASSIGNED]
MID LANE	[ASSIGNED]
BOT LANE	[ASSIGNED]
SUPPORT	[ASSIGNED]
Optimization Score: 98.5%	